

triad

Executive Summary

This module computes a reference attitude frame using the TRIAD method that aligns the spacecraft body thrust axis with a commanded inertial thrust direction, while using the Sun direction to resolve the remaining rotational degree of freedom. The TRIAD method constructs an intermediate orthonormal triad — the $\mathcal{D}$ frame, with basis vectors $\hat{d}_1, \hat{d}_2, \hat{d}_3$ — whose second basis vector is the thrust direction. The $\mathcal{D}$ frame is built twice, once in body coordinates and once in inertial coordinates, and the rotation that maps between the two representations is the commanded attitude $[\mathcal{RN}]$.

Specifically, the body thrust direction ${}^B\hat{t}$ (read from `bodyHeadingInMsg`) is matched exactly to the inertial thrust reference ${}^N\hat{t}_{\text{ref}}$ (a configuration parameter). The Sun direction ${}^N\hat{r}_{S/B}$ (from `attNavInMsg`) and the solar array drive axis ${}^B\hat{a}$ (configuration parameter) are used to construct the auxiliary triad axes that fix the spin about the thrust direction. The goal is to place the solar array drive axis orthogonal to the Sun direction with the remaining rotational degree of freedom. This constraint can only be perfectly met when the solar array drive axis is orthogonal to the thrust body axis, or when the inertial Sun direction is orthogonal to the inertial thrust reference direction. The triad frame is illustrated below.

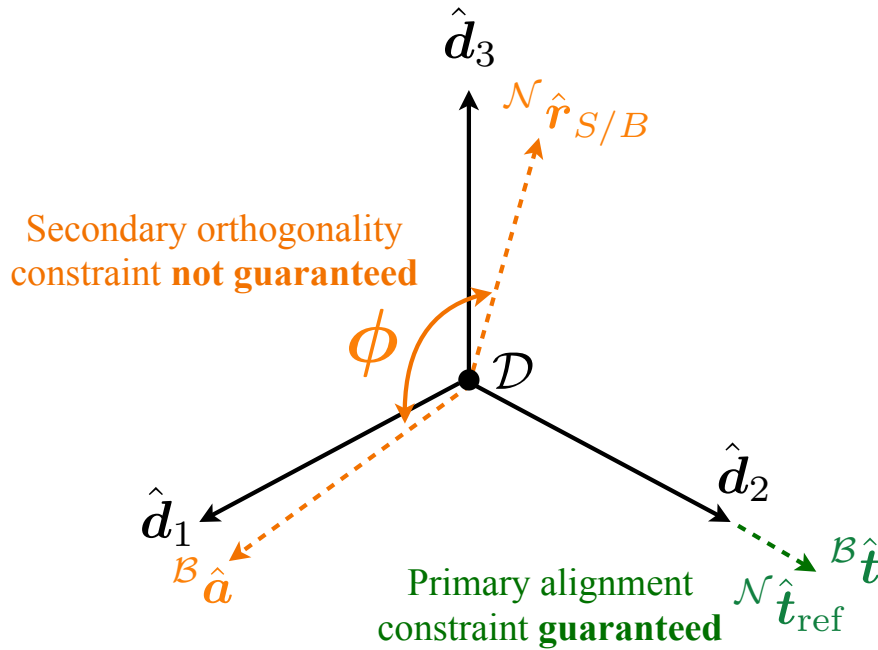


Figure 1: Illustration of the TRIAD reference frame construction

The mathematical details can be found in R. Calaon's PhD thesis, "Guidance, Control and Momentum Management of Spacecraft with Multiple Pointing Constraints". This is a single-precision (float32) port of the original double-precision Xmera implementation.

Module Architecture

The module is split into two layers:

- The **adapter** (`triad.h/.cpp`) is the SysModel-derived class that handles message I/O, validates

configuration, builds an immutable `TriadConfig` from public properties, and constructs the algorithm via two-phase initialization.

- The **algorithm** (`triadAlgorithm.h/.cpp`) is a pure C++23 class with no framework dependencies. It takes message payloads as input, computes the attitude reference, and returns a payload struct as output. It must not throw from `update()`.

A pure-C shim (`triadAlgorithm_c.h/.cpp`) wraps the algorithm class for use by Ada/Adamant components via `extern "C"` bindings.

Adapter Layer

The adapter consumes the following messages and public configuration properties:

Table 1: Module I/O Messages

Msg Variable Name	Msg Type	Description
<code>attNavInMsg</code>	<code>NavAttMsgF32Payload</code>	Navigation attitude input (uses <code>sigma_BN</code> and <code>vehSunPntBdy</code> as ${}^B\hat{r}_{S/B}$)
<code>bodyHeadingInMsg</code>	<code>BodyHeadingMsgF32Payload</code>	Body thrust direction input (uses <code>rHat_XB_B</code> as ${}^B\hat{t}$)
<code>attRefOutMsg</code>	<code>AttRefMsgF32Payload</code>	Commanded attitude reference output (writes <code>sigma_RN</code>)

Table 2: Module Configuration Properties

Parameter Name	Type	Units	Default	Description	Bounds
<code>sadaHat_B</code> (required)	<code>Eigen::Vector3f</code>		zero	Solar array drive axis in body-frame coordinates ${}^B\hat{a}$	Must be a unit vector (norm within 10^{-3} of 1); normalized on construction
<code>thrustReqHat_N</code> (required)	<code>Eigen::Vector3f</code>		zero	Inertial thrust reference direction ${}^N\hat{t}_{\text{ref}}$	Must be a unit vector (norm within 10^{-3} of 1); normalized on construction
<code>n3Axis</code>	<code>N3Axis</code> (enum)	<code>[-]</code>	<code>plusZHat_N</code>	Selects the inertial Z-axis direction ($+\hat{n}_3$ or $-\hat{n}_3$) used in the fallback computation of the first triad axis when the Sun direction and the inertial thrust reference are nearly parallel	Must be <code>N3Axis::plusZHat_N</code> or <code>N3Axis::minusZHat_N</code>

Algorithm Layer

Mathematical Formulation

Using the input messages and configuration parameters, the TRIAD algorithm constructs the commanded spacecraft attitude $[\mathcal{R}]$ by building two orthonormal triad $\mathcal{D}$ frames — one in body frame $\mathcal{B}$ coordinates and one in inertial frame $\mathcal{N}$ coordinates.

Edge Case Guard 1

Before constructing the triad frames, the algorithm first checks for three unallowable edge cases. For all edge cases, a zero MRP (identity $[\mathcal{RN}]$) is returned.

- Edge Case 1: Zero incoming body thrust direction ${}^{\mathcal{B}}\hat{t}$.
- Edge Case 2: Zero incoming Sun direction ${}^{\mathcal{N}}\hat{r}_{S/B}$.
- Edge Case 3: Solar array drive axis ${}^{\mathcal{B}}\hat{a}$ and incoming thrust direction ${}^{\mathcal{B}}\hat{t}$ are nearly parallel.

Triggered when the angle between the solar array drive axis and the body thrust direction is less than a 5 degree threshold `kParallelThresholdRad`.

$$\theta = \arccos(|{}^{\mathcal{B}}\hat{a} \cdot {}^{\mathcal{B}}\hat{t}|)$$

Important

Note that a fundamental assumption of the algorithm is that the thrust vector operates primarily in the plane normal to the solar array drive axis, making alignment between these vectors physically impossible.

Triad Frame in Body Coordinates

The first triad frame is expressed in spacecraft body frame $\mathcal{B}$ components. The frame is built from the body thrust direction and the solar array drive axis:

$$\begin{aligned} {}^{\mathcal{B}}\hat{d}_2 &= {}^{\mathcal{B}}\hat{t} \\ {}^{\mathcal{B}}\hat{d}_3 &= \frac{{}^{\mathcal{B}}\hat{a} \times {}^{\mathcal{B}}\hat{d}_2}{\|{}^{\mathcal{B}}\hat{a} \times {}^{\mathcal{B}}\hat{d}_2\|} \\ {}^{\mathcal{B}}\hat{d}_1 &= {}^{\mathcal{B}}\hat{d}_2 \times {}^{\mathcal{B}}\hat{d}_3 \end{aligned}$$

Each axis is normalized. The matrix $[\mathcal{BD}]$ is constructed using the triad frame axes as columns: $[\mathcal{BD}] = [{}^{\mathcal{B}}\hat{d}_1, {}^{\mathcal{B}}\hat{d}_2, {}^{\mathcal{B}}\hat{d}_3]$.

Triad Frame in Inertial Coordinates

The second triad frame is expressed in inertial frame $\mathcal{N}$ components. The frame is build from the reference inertial thrust direction and the inertial Sun direction:

$$\begin{aligned} {}^{\mathcal{N}}\hat{d}_2 &= {}^{\mathcal{N}}\hat{t}_{\text{ref}} \\ {}^{\mathcal{N}}\hat{d}_1 &= \frac{{}^{\mathcal{N}}\hat{r}_{S/B} \times {}^{\mathcal{N}}\hat{d}_2}{\|{}^{\mathcal{N}}\hat{r}_{S/B} \times {}^{\mathcal{N}}\hat{d}_2\|} \\ {}^{\mathcal{N}}\hat{d}_3 &= {}^{\mathcal{N}}\hat{d}_1 \times {}^{\mathcal{N}}\hat{d}_2 \end{aligned}$$

Each axis is normalized. The matrix $[\mathcal{ND}]$ is constructed using the triad frame axes as columns: $[\mathcal{ND}] = [{}^{\mathcal{N}}\hat{d}_1, {}^{\mathcal{N}}\hat{d}_2, {}^{\mathcal{N}}\hat{d}_3]$.

Edge Case Guard 2

Before constructing the second triad frame, the algorithm checks whether the Sun direction and the thrust inertial reference direction are nearly parallel using the same 5 degree threshold `kParallelThresholdRad`.

$$\theta = \arccos(|{}^{\mathcal{N}}\hat{r}_{S/B} \cdot {}^{\mathcal{N}}\hat{t}_{\text{ref}}|)$$

The second triad frame is ill-defined when these vectors are parallel. In this case, we instead seek to align the first triad axis with the positive or negative inertial frame Z-axis, depending on the configured `n3Axis` parameter. To do so, ${}^{\mathcal{N}}\hat{d}_3$ is instead computed as the cross product between the thrust reference and the inertial Z-axis. Crossing the second triad axis with the new third triad axis ensures the first triad axis will align as close to the inertial Z-axis as possible.

$$\begin{aligned}\mathcal{N}\hat{d}_2 &= \mathcal{N}\hat{t}_{\text{ref}} \\ \mathcal{N}\hat{d}_3 &= \frac{\pm \mathcal{N}\hat{n}_3 \times \mathcal{N}\hat{t}_{\text{ref}}}{\|\pm \mathcal{N}\hat{n}_3 \times \mathcal{N}\hat{t}_{\text{ref}}\|} \\ \mathcal{N}\hat{d}_1 &= \mathcal{N}\hat{d}_2 \times \mathcal{N}\hat{d}_3\end{aligned}$$

Important

Because the spacecraft trajectory never passes directly beneath the Sun, the new cross products should never be zero. However, in the case where the thrust reference and sun direction are both nearly parallel to the fallback Z-axis, the new cross products will be zero. The current spacecraft attitude is returned in this case because this is an impossible configuration.

Commanded Attitude

The commanded body frame reference attitude is simply the composition:

$$[\mathcal{RN}] = [\mathcal{BN}] = [\mathcal{BD}] [\mathcal{ND}]^T$$

where $[\mathcal{RN}] = [\mathcal{BN}]$ because the spacecraft body frame $\mathcal{B}$ is ultimately driven to the reference frame $\mathcal{R}$. The DCM result is converted to an MRP $\sigma_{\mathcal{R}/\mathcal{N}}$ and returned.

Algorithm Assumptions and Limitations

- If either the incoming Sun direction vector or body thrust direction are zero, the algorithm returns the zero MRP (identity attitude).
- The body thrust vector operates primarily in the plane normal to the solar array drive axis, making alignment with the solar array drive axis physically impossible. If the solar array drive axis is near parallel to the body thrust direction, the algorithm returns the zero MRP (identity attitude).
- If the inertial Sun direction and the thrust inertial reference direction are nearly parallel, the second triad frame is ill-defined. In this case, the first and third inertial triad axes are computed using a fallback configured inertial Z-axis direction. The zero MRP (identity attitude) is returned in the case where the fallback Z-axis is also nearly parallel to both the Sun direction and the thrust inertial reference direction.

Algorithm Visualization

Recall that the triad frame construction does not guarantee that the Sun-SADA axis orthogonality constraint is met unless either (1) the SADA axis is orthogonal to the body thrust direction ${}^{\mathcal{B}}\hat{t}$ or (2) the inertial thrust reference is orthogonal to the inertial Sun direction. While there may be a configuration that better or perfectly satisfies the Sun orthogonality constraint, the triad algorithm only guarantees that the orthogonality constraint is bounded by an angle θ . The angle θ is defined as the offset of the thrust body vector from the plane normal to the SADA axis (For the EMA mission, the SADA axis is along the spacecraft +X axis and the angle θ is therefore the body thrust offset angle from the spacecraft YZ-plane). In other words, the final solar array incidence with the Sun is bounded by the offset angle θ . This fundamental property of the triad algorithm is not immediately intuitive. The important point here is that the solar array incidence in fact

cannot be equal to the offset angle θ , instead it is upper bounded by the thrust offset angle θ . A simple example illustrating this property is shown in Figure 2 below.

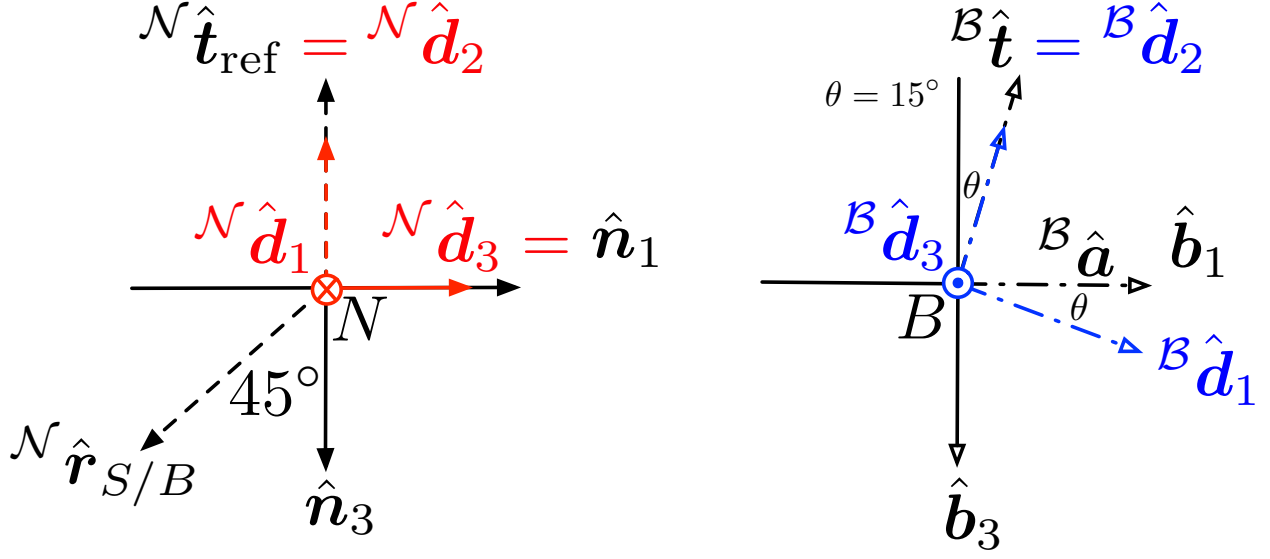


Figure 2: TRIAD algorithm example

Figure 2 presents a simple example of the triad algorithm. The left figure sets up the inertial frame, where the inertial thrust reference direction and Sun direction are defined. The right figure sets up the spacecraft body frame, where the solar array drive axis and body thrust vector are defined. The inertial triad frame construction is shown on the left figure in red, while the body frame triad construction is shown in the right figure in blue.

Viewing the figures, it is clear that in order to align the body triad frame with the inertial triad frame, the body triad frame must first be rotated by θ degrees about the third triad axis $B \hat{d}_3$. To complete the transformation, the body triad frame must then be rotated by 90 degrees about the second triad body axis $B \hat{d}_2$.

Applying this transformation to the body SADA axis gives the final SADA axis direction in inertial frame components:

$$\mathcal{N} \hat{a} = \begin{bmatrix} 0 \\ -\cos(\theta) \\ -\sin(\theta) \end{bmatrix} = \begin{bmatrix} 0 \\ -0.965926 \\ -0.258819 \end{bmatrix}$$

The final angle between the inertial SADA axis and the Sun direction can be computed:

$$\phi = \arccos(|\mathcal{N} \hat{a} \cdot \mathcal{N} \hat{r}_{S/B}|) = 100.55^\circ$$

Subtracting 90 degrees from the above result gives the Sun-array offset angle, which is 10.55 degrees and indeed less than the thrust offset angle $\theta = 15^\circ$, as expected. In the nominal case, the following condition holds and is the aforementioned fundamental property of the algorithm:

$$\theta > \phi - 90^\circ$$

Summarizing, the solar array-Sun incidence angle will be bounded by the thrust offset angle θ for nominal conditions. The nominal configuration is also illustrated in Figure 3 for clarity. Note that the incidence between the Sun direction and the inertial SADA axis is denoted ϕ .

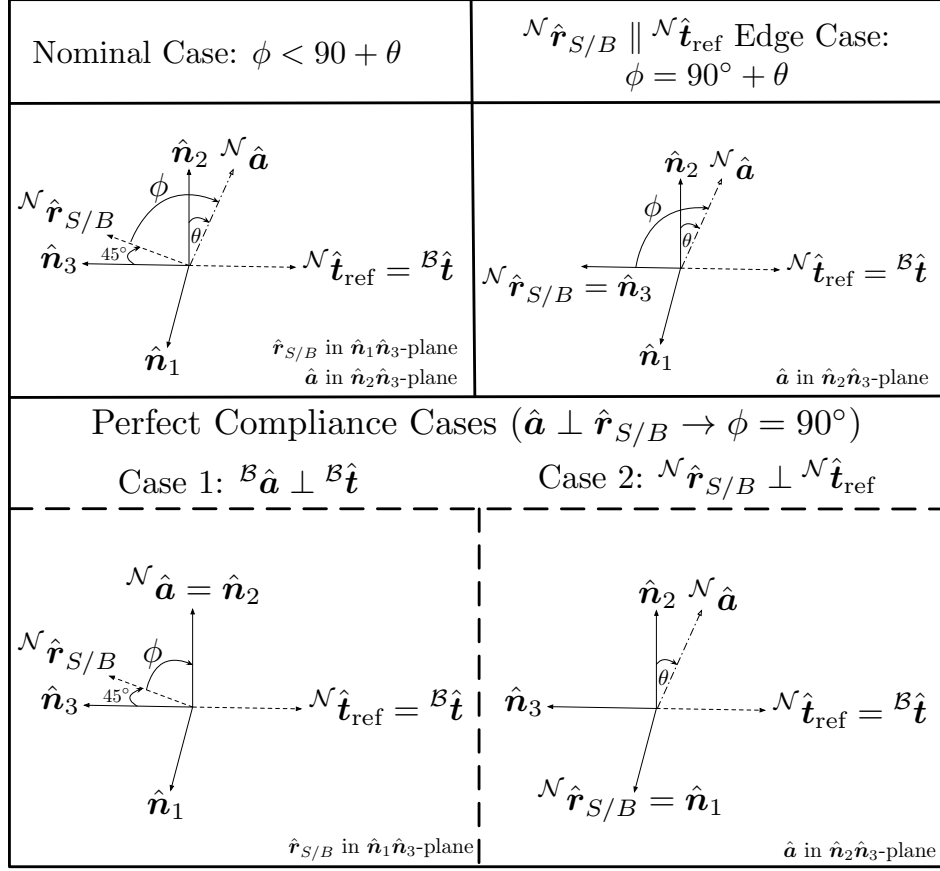


Figure 3: Set of possible TRIAD configurations.

Test Description

The module is verified through regression tests that compare the algorithm results against an independent reference implementation. Setup tests are used for the **TriadConfig** validators and round-trip tests are used to check the set configuration variables are correctly returned. Fuzz tests are added for the regression and property tests, where the configuration and inputs are randomized over reasonable ranges.

Property Tests

- **OutputIsFinite**
 - Checks that all output components are finite for valid inputs.
- **ThrustBodyHeadingAlignedToThrustInertialHeading**
 - Checks the thrust body axis aligns with the inertial heading direction (Constraint guaranteed).
- **SigmaRnNormBounded**
 - Checks that the output MRP is bounded by 1 (inner MRP set) for any inputs.
- **SolarArraySunOffsetBoundedByBodyThrustOffset** - Checks that the solar array offset from the Sun direction is bounded by the body thrust vector offset angle from the plane normal to the solar array drive axis
 - An example for this property is illustrated in Figure 3: Nominal Case

Edge Case Tests

- **ZeroThrustDirectionReturnsZero**
 - Checks that the algorithm correctly returns the zero MRP (identity attitude) when the incoming thrust direction message is zero.
- **ZeroSunDirectionReturnsZero**
 - Checks that the algorithm correctly returns the zero MRP (identity attitude) when the incoming Sun direction message is zero.
- **SadaAlignedBodyThrustReturnsZero**
 - Checks that the algorithm correctly returns the zero MRP (identity attitude) when the SADA axis is aligned with the body thrust direction.
- **SunAlignedWithThrustRefUsesFallbackZAxis**
 - Checks that when the Sun direction is aligned with the thrust inertial reference, the fallback inertial Z-axis is used in triad body frame computation.
- **SadaOrthogonalToSunWhenOrthogonalToBodyThrust**
 - Checks that the solar array-Sun orthogonality constraint is met when the SADA axis is orthogonal to the body thrust axis.
 - This test is illustrated in Figure 3: Perfect Compliance Case 1
- **SadaOrthogonalToSunWhenSunAndThrustRefAligned**
 - Checks that the solar array-Sun orthogonality constraint is met when the Sun direction is orthogonal to the inertial thrust reference direction.
 - This test is illustrated in Figure 3: Perfect Compliance Case 2
- **ReturnedOutputMatchesPrecomputedReference**
 - This test checks that the algorithm output matches a pre-computed reference (Computation done by hand). The example uses a body thrust direction defined out of the spacecraft YZ-plane.
- **InertialSadaAxisMatchesPrecomputedAxis**
 - This test uses the algorithm output to check that the final inertial SADA axis matches the pre-computed SADA axis direction (Computation done by hand).